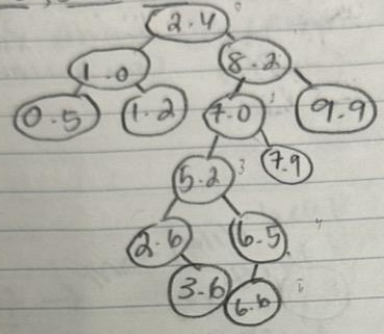
# Assignment 6

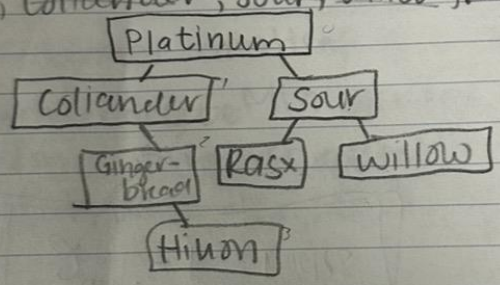
## #1 Type of Tree

a] 2.4, 8.2, 7.0, 5.2, 1.0, 6.5, 2.6, 3.6, 7.9, 0.5, 6.6, 9.9, 1.2



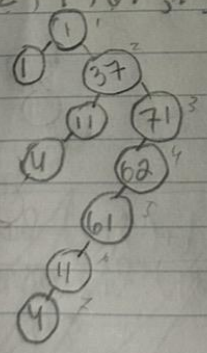

Height = 5

b] Platinum, Coliander, Sour, Willow, Rasx, Gingerbread, Hivon




Height = 3

c) 1, 37, 11, 71, 62, 4, 61, 1, 11, 4

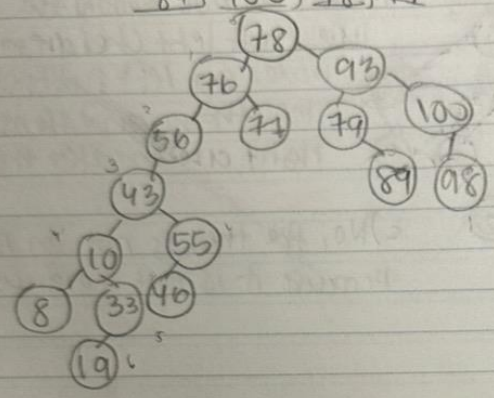



Height = 7

Tree rules defined as
≤ on left
and > on right

#1 (Cont.)

d) 78, 76, 56, 43, 55, 77, 10, 93, 46, 79, 33, 8, 89, 100, 98, 19

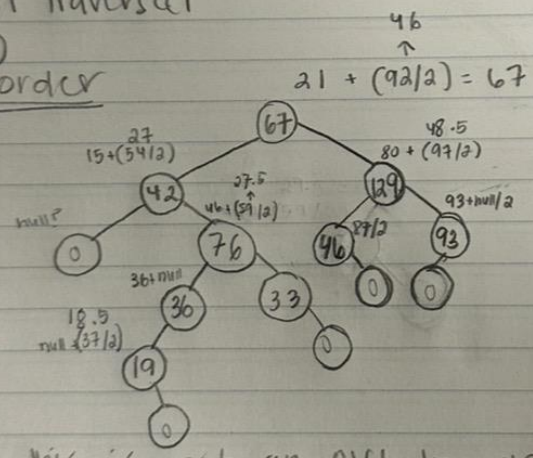



Height = 6

#2 BST Traversal

a1)

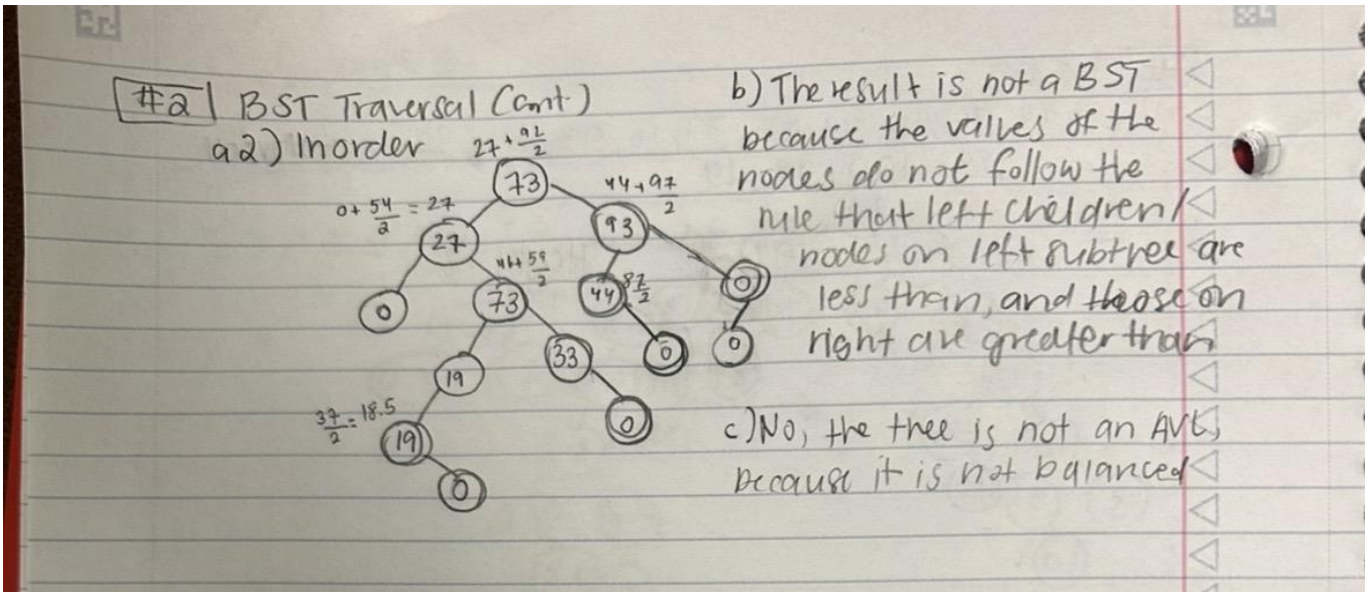
preorder




b) No, this is not a BST, because node values left to some nodes are greater than, and some to right of some nodes are less than

↳ ex) 76 is to left of 67
33 is right of 76

c) No, this is not an AVL because the tree is not balanced.




**7) Algorithm Analysis**

#3. The time complexity for the total encoding algorithm will be broken down (then put back together). Worst case and best case would be the same.

- FrequencyCount: time complexity is O(N+M) (M could be dropped here since it is bounded by N) where N is the length of the input string and M is the number of non-repeating/unique/single characters (at most N) in the string because the loop iterates through the string (N) and then converts the list (that is non-repeating characters—M) into nodes. Because these are independent the final time compelxity is O(M+N). Space is O(M) because table and list created are of length M.
- BuildHuffman: time complexity is O(M^2logM) because the loop runs M-1 times, but the sorting costs MlogM. The sorting happens every iterations, so the complexities are multiplied. The space complexity O(M) because we are creating the left, right, and parent nodes for all nodes (M).
- Gencode: time complexity is O(M) because it traverses the entire tree once (M). The space complexity of O(1) because all operations are constant
- Huffencoding: time complexity is O(N + M^2logM) because we are using frequency count, buildHuffman, and genCode. The space complexity is O(M) because we are using those methods.
- Encode: the time complexity is O(N+M^2logM) because we are using huffencoding and we are also iterating through a string (O{N}), this can simplify to final complexity. The space complexity would be O(M) because we are using huffEncoding .

So, for Huffman, time complexity would be O(N+M^2logM).

#4. Encrypted Binary Tree – The time complexity is O(N), where N is the total number of nodes in the tree because the for loop iterates the length of the inputted array (which is N) (this makes sense because we have to check each node to see if it is the target node). Additionally, all other operations are in constant time. Omega(1) because the root could be the target. The space complexity is O(N), where N is the total number of nodes in the tree because the array "actual" is the length of the inputted array (which is N). Omega(1), where array of length one is created. All other declarations are constant.

#5. All Less – The time complexity is O(N) where N is the length of the inputted array of strings because there is a single for-loop that iterates N times. Other methods, like .length (for an array) or .length() for a string are both constant because length is member data for arrays and length is found similarly for strings (where string is created with certain length, that can then be accessed). All other operations are comparisons or declarations, which are constant. Omega(N) because you still must check each word. The space complexity is O(M) where M is the total number of characters in the inputted array because the string "list" is created, however its length is based on the amount of strings in s that have length less than x. In the worst case scenario, all elements' lengths would be less than x, so list would be made up of all strings in the array (so length of list would be all characters in array. Omega(1) where no strings are added to the array.

#6. The time complexity would be O(logN * M) where N is the number of nodes and M is the number of nodes that need to be removed. The code represents an implementation of a BR tree, thus the worst case for removing a node would be O(logN) (as discussed in class) times the number of nodes removed (M). The best case scenario would be if the node to be removed is a red leaf because there is no need for recoloring and rotations (though you still need to search first). The space complexity would be O(logN) because i am using a stack to keep track to what to remove, and the space being used is proportional to height.